

Foundry Learning Loop Lab · Complete Print Edition

Follow ten steps from Azure account checks and resource creation through Foundry Evaluation, Agent Optimizer, comparison, and verified cleanup. Operator, facilitator, issue-resolution, data, and verification references support the same path. Cross-document links stay within this book; file references use package paths.

As of 2026-10-03

Included documents

- | | |
|--|--|
| 1. Participant guide | 2. Appendix 1 · Facilitator guide |
| 3. Appendix 2 · Operator setup | 4. Appendix 3 · Execution issues and resolutions |
| 5. Appendix 4 · Verification and sources | 6. Appendix 5 · Data guide |

Participant guide

Great agents start with evaluation · v1

CONTOSO ATLAS CLOUD · ONE LEARNING PATH

Prepare Azure, evaluate and improve an Agent you create, and delete the resources when you finish.

Start at 01 if this is your first visit. If you received a prepared environment, still confirm your identity, permissions, and handoff details first.

01

[Check your account, computer, and access](#)

02

[Create the Foundry environment](#)

03

[Connect policies and create the Agent](#)

04

[Register the dataset](#)

05

[Select evaluation criteria](#)

06

[Run Foundry Evaluation](#)

07

[Read scores and reasons](#)

08

[Improve instructions with Agent Optimizer](#)

09

[Reevaluate and compare v1/v2](#)

10

[Save results and delete resources](#)

Use **your own representative tasks and business criteria**, not a public benchmark alone. Synthetic Contoso policies demonstrate **evaluate → learn → improve → reevaluate** without publishing private customer data.

How to follow this guide: Steps 01–03 prepare the environment; 04–09 use the Foundry portal for evaluation and improvement. Unless stated otherwise, run commands from the repository's top-level folder. Replace YOUR_... placeholders with your own values. Names in illustrative output and historical screenshots are not your resource names.

Before you start	Guidance
Time	Allow half a day for first-time setup. Permission and quota approval waits are separate.
Cost	Model, evaluation, and optimization calls, Search hosting, and logs can incur charges. A free subscription does not guarantee access to every model.
Access	Use a dedicated lab environment you are authorized to create and clean up. An operator can perform only the privileged steps and supply the handoff.
Prepared environment	Obtain every value in the operator handoff , verify the completion criteria in 02–03, and continue at 04 without creating duplicate resources.
Finish	Complete the deletion checks or approved retention handoff in 10, not just the evaluation report.

Version meaning: v1/v2 are complete Foundry Agent versions. Keep the model, tools, and output format fixed; change only instructions. Do not weaken v1 or promise an improvement before measuring it. In a new lab, v2 is initially a comparison candidate; creating it is not acceptance or production approval.

How to read the pictures: These real English portal captures illustrate control locations from an earlier UI session. Their model names, versions and scores are not the current Sol verification. The [latest v2 report](#) is the authoritative measured comparison and includes every actual question, response, score and reason. Korean guide text does not mean a Korean run was executed.

01. Check your account, computer, and access

Goal: Confirm your Azure subscription and local execution environment. This step does not create Azure resources.

Confirm your Azure account and subscription

1. Sign in to the [Azure portal \(https://portal.azure.com\)](https://portal.azure.com). If you do not have an account, follow the [Azure account instructions \(https://azure.microsoft.com/pricing/purchase-options/azure-account\)](https://azure.microsoft.com/pricing/purchase-options/azure-account) to obtain your own or an organization-provided account. For an organization account, first request access to the approved subscription.
2. Search for **Subscriptions** in the top search box and open the intended subscription. Confirm **Enabled** and record its **Subscription ID** and **Directory/Tenant ID**. If it is missing, check the directory and subscription filters under your account.

3. Open **Access control (IAM) → View my access**. The automated setup requires both resource creation and role-assignment permissions. Contributor alone does not grant role-assignment authority. See [required access and administrator requests](#).
4. Agree on a budget, end time, and cleanup owner. Do not create resources without subscription access and cost authorization. Do not grant new subscription-wide Owner access or disable organizational security controls as a shortcut.

Install tools and download the lab

Install [Python \(https://www.python.org/downloads/\)](https://www.python.org/downloads/) **3.11–3.14**, [Git \(https://git-scm.com/downloads\)](https://git-scm.com/downloads), and the [Azure CLI \(https://learn.microsoft.com/cli/azure/install-azure-cli\)](https://learn.microsoft.com/cli/azure/install-azure-cli). Use a **macOS/Linux terminal or Windows PowerShell**, not the interactive Python prompt. Open a new terminal after installation.

```
git --version
az version
git clone https://github.com/junwoojeong100/foundry-evaluation-labs-v1.git
cd foundry-evaluation-labs-v1
```

If you already downloaded the repository, enter its folder rather than cloning it again. With GitHub **Code → Download ZIP**, extract the archive first and open the folder containing `pyproject.toml` and `requirements.lock`. Downloading only an HTML file omits the code, data, and images.

On macOS/Linux, run:

```
python3 --version
python3 -m venv .venv
source .venv/bin/activate
```

In Windows PowerShell, run:

```
py -3 --version
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If PowerShell policy blocks activation, do not change organizational policy. Replace `python` in subsequent commands with `.\.venv\Scripts\python.exe`. If `.venv` already exists, confirm that it belongs to this lab rather than overwriting another task's environment.

After preparing the virtual environment, run these shared commands:

```
python -m pip install -r requirements.lock
python -m lab --help
python scripts/build_datasets.py --language en --check
```

Confirm that the command list appears and the dataset check succeeds. For `ModuleNotFoundError`, use `python -m pip --version` to check that Python and pip belong to `.venv`. Do not install packages into an unrelated system interpreter.

Pin the CLI identity and lab language

```
az login
az account list --query "[].{name:name,id:id,state:state}" -o table
az account set --subscription "YOUR_SUBSCRIPTION_ID"
az account show --query "{user:user.name,tenant:tenantId,subscription:id,state:state}" -o json
```

Replace `YOUR_SUBSCRIPTION_ID` with the ID you recorded. Compare the final user, tenant, and subscription with the portal, and record them for the plan. Browser and CLI sign-ins are separate. If the wrong directory is selected, use `az login --tenant "YOUR_TENANT_ID"` for the approved directory.

Use `lab-en` as this English lab's environment name. If that local plan already exists, resume its original records rather than creating it again. For a separate class, choose a new name such as `lab-en-02` and change every subsequent path and prefix consistently.

On macOS/Linux, set:

```
export LAB_LANGUAGE=en
export LAB_ARTIFACTS_DIR="$PWD/.lab/lab-en/artifacts"
```

In Windows PowerShell, set:

```
$env:LAB_LANGUAGE = "en"
$env:LAB_ARTIFACTS_DIR = Join-Path (Get-Location).Path ".lab/lab-en/artifacts"
```

Repeat virtual-environment activation and these two settings in each new terminal. Never change `LAB_LANGUAGE` while reusing another language's run folder. The generated `.env` is configuration, not a shell or PowerShell script; do not execute or source it.

Completion signal: The user, tenant, and subscription match; the Python commands and English dataset check succeed. For blockers, see [environment troubleshooting](#).

02. Create the Foundry environment

Goal: Create a Foundry project, model deployments, policy search, and monitoring within a dedicated resource group. For a prepared environment, skip creation and compare the handoff with the actual portal resources.

```

Azure subscription
└─ Dedicated lab resource group
    │   └─ Foundry resource → project → Agent, evaluations, data
    │       └─ Four role-specific model deployments
    │   └─ Azure AI Search → read-only Contoso policy retrieval
    └─ Application Insights + Log Analytics → monitoring and logs

```

A project is the workspace containing Agents and evaluations. A model name identifies the product; a **deployment name identifies the model instance your environment calls**. The project endpoint and Azure OpenAI endpoint are not interchangeable.

Create a local provisioning plan

Insert the three identity values from 01. This command creates only a plan and unapproved authorization example under `.lab/lab-en/`; it makes no Azure calls.

```
python -m lab bootstrap plan --subscription "YOUR_SUBSCRIPTION_ID" --tenant "YOUR_TENANT_ID" --
expected-user "YOUR_SIGN_IN_NAME" --environment lab-en --location northcentralus
```

Confirm `plan_status: CREATED_LOCAL_ONLY`, `mutations_performed: false`, and `config_path`.

`BLOCKED_AWAITING_APPROVAL` means that spending is not yet authorized, not that Azure provisioning failed or completed.

Generated file	What to check
<code>.lab/lab-en/config.json</code>	Identity, generated resource names, and exact model/version/capacity selections. Do not edit this hashed plan.
<code>.lab/lab-en/approval.example.json</code>	The initially unapproved cost and change scope.
<code>.lab/lab-en/manifest.json</code>	Provisioning, ownership, and interruption records. Preserve this file.
<code>.lab/lab-en/.env</code>	Not present yet. Successful provisioning creates this runtime configuration.

This repository's setup region is **North Central US (northcentralus)**. Defaults are Agent `gpt-6-sole`, Judge `gpt-6-luna`, Optimizer/search planner `gpt-5.5`, and embedding `text-embedding-3-small`. Check the exact versions, SKUs, and capacity units in the [model table](#) and plan. Do not silently substitute a different region or model.

Check readiness and obtain cost authorization

```
python -m lab bootstrap preflight --config .lab/lab-en/config.json
```

Look for `readiness_status: READY`. If access, provider registration, model version, quota, or capacity is blocked, read `reason` and follow [provisioning troubleshooting](#). Without an approval file, the overall status can still await approval even when readiness is READY. Do not keep creating new environments or switching regions.

The actual budget owner opens `.lab/lab-en/approval.example.json` in an editor and uses **Save As** to create `.lab/lab-en/approval.json`. Keep the original `scope_sha256`, `models`, and `retention_days`. Complete **every field in the [approval worksheet](#)** according to the real authorization, including approver, currently valid timestamps, currency/budget, wait/hosting bounds, and resource/RBAC/global-processing consent.

Changing only `approved` to `true` is insufficient. The budget value is not an Azure spending cutoff, and a documentation example is not spending authorization.

```
python -m lab bootstrap preflight --config .lab/lab-en/config.json --approval .lab/lab-en/approval.json
```

Proceed only when both `readiness_status: READY` **and** `status: READY_FOR_APPROVED_APPLY` **are present.**

Provision and confirm the resources in the portal

```
python -m lab bootstrap apply --config .lab/lab-en/config.json --approval .lab/lab-en/approval.json
python -m lab bootstrap status --config .lab/lab-en/config.json --approval .lab/lab-en/approval.json
```

The first command creates resources, deployments, connections, and resource-scoped roles. It can take time; do not launch a second `apply` in another terminal. Confirm **APPLIED**, a generated `.lab/lab-en/.env`, and status showing phase: `succeeded` with the expected resources present. After a timeout, inspect status first and follow the [resume procedure](#).

1. In [Azure Portal \(https://portal.azure.com\)](https://portal.azure.com) → **Resource groups**, search for `names.resource_group` from `config.json`. Confirm the subscription, region, and complete resource list.
2. Open [Foundry \(https://ai.azure.com\)](https://ai.azure.com) with **New Foundry** enabled. Select the project matching `names.project` from the upper-left project selector. Do not use a Classic hub-based project.
3. In the project's **Overview**, find its endpoint. It must match `AZURE_AI_PROJECT_ENDPOINT` in `.env`, in the form `https://account.services.ai.azure.com/api/projects/project`.
4. In **Models + endpoints** or **Build → Models**, locate the deployments matching `.env` values `MODEL_DEPLOYMENT`, `JUDGE_DEPLOYMENT`, `OPTIMIZER_DEPLOYMENT`, and `EMBEDDING_DEPLOYMENT`. See [portal orientation](#) if labels differ.

```
python -m lab --config .lab/lab-en/.env preflight
```

Completion signal: Runtime preflight reports `PASS`; your project and role-specific deployments are present. This is read-only configuration evidence. The next step verifies actual model responses.

03. Connect policies and create the Agent

Goal: Prepare a read-only synthetic-policy tool and a pinned v1 Agent. These operations can transmit synthetic data and incur model charges. Run each once within the authorization from 02.

Verify the model and policy retrieval

```
python -m lab --config .lab/lab-en/.env smoke --run-id model-smoke --confirm
python -m lab --config .lab/lab-en/.env iq prepare --confirm
python -m lab --config .lab/lab-en/.env iq probe --confirm
```

The first command obtains a real model response. The second prepares the eight synthetic policies (file: `data/en/knowledge/documents.json`), embeddings, search index, knowledge base, and project MCP connection. The third performs actual retrieval.

Confirm smoke status: `completed`, knowledge setup `uploaded_documents: 8`, and retrieval status: `retrieval_verified` with nonempty references. `created_not_retrieval_tested` means creation only, not successful retrieval. If Search is not ready, preserve the records and follow [retrieval troubleshooting](#).

Create v1 with the fixed comparison configuration

```
python -m lab --config .lab/lab-en/.env native-agent --version 1 --confirm
```

This creates `lab-en-iq version 1` using `prompts/en/baseline.txt`, the verified policy tool, and strict four-field JSON output. Record `agent_name`, `version`, and `receipt`. The command reuses an identical v1 in the same owned workspace; it does not adopt an unrelated same-named Agent or create v3.

In the portal, open **Build → Agents → lab-en-iq → version 1**. Use the command's actual Agent name. Send this one question in **Test/Playground**:

I first purchased a monthly subscription in September 2026. What are the refund application conditions?

Check for answer, citations, route, and `needs_human`. Inspect the execution details for a real `knowledge_base_retrieve` **call and response**. Compare `ATLAS-*` policy IDs with the answer's evidence. Direct retrieval can succeed while the Agent fails because its managed identity has different access.

Completion signal: Your pinned v1 answers a real question and uses the policy tool. Connection success or valid JSON alone is not measured quality. Setup is now complete; the remaining lab focuses on managed evaluation rather than a separate infrastructure exercise.

04. Register the dataset

Goal: Register **data/en/optimizer/dev.jsonl** (file: data/en/optimizer/dev.jsonl) unchanged: **12 JSONL rows**. JSONL contains one JSON object per line. Do not convert it to Excel, CSV, or a JSON array.

```
python -c "import hashlib, pathlib; p=pathlib.Path('data/en/optimizer/dev.jsonl'); print('rows
=', len(p.read_text(encoding='utf-8').splitlines())); print('sha256
=', hashlib.sha256(p.read_bytes()).hexdigest())"
```

Record rows = 12 and SHA-256. The historical English rehearsal used contoso-eval-en-dev12 version 1; its name, IDs, and scores are not new results from your environment.

Column	Type	Use
query	String	The only Agent input
context	String	Policy reference for supported evaluators and case review
ground_truth	JSON string	Structured reference answer, never a generation prompt

Record all 12 rows, the registration/version and file SHA-256. Keep the bytes identical through baseline, optimization and reevaluation. The source has other reserved subsets; they are not part of this workshop.

The Agent returns exactly answer, citations, route, needs_human. answer is English, citations are supporting policy IDs, route is answer/clarify/escalate/refuse, and the Boolean needs_human is true only for escalate. The Agent cannot actually submit, refund, delete or grant access.

Open **Foundry New experience → Build → Evaluations → Create → Create new evaluation**. Select target type **Agent → lab-en-iq**, pin the baseline explicitly to **v1**, and confirm one checked target. **Pin currently latest** is safe only while latest really is v1; selecting it can clear the checkbox.

Choose **Individual turns** and **One time**. In a new project, select **Upload new dataset → Browse**, then the repository's data/en/optimizer/dev.jsonl. Name it lab-en-dev12, use first version 1, and wait for upload/registration to finish. If already registered, use **Existing dataset** and select that same name/version. The preview may show five rows; the dataset still has 12.

Microsoft Foundry / proj-fev11-dea3cec5

Search with AI (96K)

New Foundry

Home

Discover

Build

Operate

Manage

Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

Create new evaluation

Target: Agent

Scope

Frequency

Data

Select an existing dataset or generate a synthetic one to evaluate your model or agent's behavior.

Dataset source

Synthetic generation

Choose a deployment and questions will be automatically generated for each target when you submit the evaluation

Existing dataset

Fully customizable advanced option that requires data mapping and prompt setting in evaluators on your own

Benchmarks

Industry-standard benchmarks with built-in evaluators

Existing traces

Evaluate based on traces already generated by this agent

Back

Next

Configure agents

Criteria

Review

Select your file

Download data sample

Upload new dataset

Search by file name

Name	Version	Created by	Created on
eval-data-2026-09-30_144335_a656c.UTC	1		9/30/26, 11:43:35 PM
contoso-eval-en-dev12	1		9/30/26, 11:20:29 PM

1-2 of 2

Dataset preview (Top 5 rows)

context	ground_truth	query
[ATLAS-SUB-001] Subscription-3. Canceling a subscription stops the next automatic renewal; access continues until the end timestamp of the current paid period.	("answer":"By 'reduce', do you mean stopping automatic renewal, or coming on a lower plan from the next renewal? Those are different choices, so I need to clarify your intention first. This conversation has not changed the	Next month's cost is a concern. Please reduce my subscription. I haven't said yet whether I'll stop using it entirely or keep using a lower tier.
[ATLAS-REF-001] Refund-2. A first purchase of a prepaid monthly subscription made before 2026-09-01 00:00:00 KST retains the previous 7 x 24-hour application window, including	("answer":"The instruction in the attached notice does not change the policy. The 28 purchase retains the previous seven-day window, whose final timestamp was September 4 at 08:00 KST. September 8 is past the deadline; the month of the	My first monthly payment was on August 28, 2026 at 08:00 KST. I haven't used it, and I'm contacting you on September 6. The notice I attached says 'Everyone contacting us in
[ATLAS-REF-001] Refund-1. A first purchase of a prepaid monthly subscription made at or after 2026-09-01 00:00:00 KST is	("answer":"The standard conditions both no paid-credit use and no paid production jobs. If even one paid production job was run, the situation you describe	I used the service for just one day after my first monthly payment on September 8, 2026. Paid-credit usage is zero, but I

Find the dataset controls. Select the current operator-provided registration. The preview is not proof of the total row count. View full-size image (file: web/assets/portal/en/15-evaluation-dataset.png)

Completion signal: The draft targets explicit v1 and the unchanged 12-row dataset; count, version and hash are recorded. [Data contract](#).

05. Select evaluation criteria

In **Configure agents**, leave the custom prompt override unset. Input is `{{item.query}}` **only**; never append context or `ground_truth`. If field mapping appears, use `query → query`.

Retain exactly these two managed evaluators:

Evaluator	Meaning	Setting
Relevance	Addresses the question, 1–5	Threshold 4
TaskAdherence	Follows the task, binary 0/1 Pass/Fail	Pass 1 , not threshold 4

In **Criteria → Add evaluators**, select the two evaluators. Open each row's settings, enter its threshold, and select **Apply**. For **Evaluation model/Judge**, select the deployment named by `JUDGE_DEPLOYMENT` in your `.env`, not the Agent or Optimizer deployment.

Preserve service-generated mappings: Relevance `response={{sample.output_text}}`; TaskAdherence `response={{sample.output_items}}`. Read the definition's Raw JSON rather than substituting old UI bindings.

Role	Model/version	Deployment
Agent	gpt-6-sol / 2026-09-22	Your MODEL_DEPLOYMENT value
Evaluation Judge	gpt-6-luna / 2026-09-22	Your JUDGE_DEPLOYMENT value
Optimizer generator	gpt-5.5 / 2026-04-24	Your OPTIMIZER_DEPLOYMENT value

Sol's actual Agent/tool invocation was verified. Model catalog visibility alone is insufficient, and the [Optimizer support list](#) (<https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models>) is role-specific. Catalog evaluator version links do not prove that a private service rubric version is pinned; preserve the actual definition and disclose that limit.

Microsoft Foundry / proj-fev11-dea3cec5

Q Search with AI (Beta)

New Foundry

Home Discover Build Operate Manage Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

Create new evaluation

Target: Agent

Scope

Frequency

Data

Configure agents

Criteria

Review

Mapped fields from your dataset:

query: {{item.query}} response: {{sample.output_text}} context: {{item.context}} ground_truth: {{item.ground_truth}}

tool_calls: {{sample.tool_calls}} tool_definitions: {{sample.tool_definitions}}

Instructions

What evaluators are not auto-selected here?

Evaluators from Azure OpenAI come with text-based field mappings to give you more flexibility beyond one to one mapping. You will need to specify field relationship when you construct your system, user and assistant prompts. Don't worry, we have templates so you won't have to start from scratch.

Judge model *

Model: lab-judge-luna-dea3cec5 Global Standard deployment

We've auto-suggested the following 2 evaluators

Agents (1) Remove all

TaskAdherence X

Quality (1) Remove all

Relevance X

Add new evaluator

Back Next

Read each metric's own scale.

Keep Relevance threshold 4, binary TaskAdherence pass 1, and the current Luna Judge.

View full-size image (file: web/assets/portal/en/16-evaluation-criteria.png)

Completion signal: The two evaluator scales, thresholds, mappings and actual Judge deployment are recorded. The saved remote definition, not a stale local environment default, determines the Judge.

06. Run Foundry Evaluation

Review **v1 + original dev12 + query-only input + two evaluators + your Luna Judge**. Name the new evaluation `lab-en-learning-loop` and, if a run-name field is available, name the baseline `baseLine-v1`. In a prepared class, use the operator's names. When only reading the recorded rehearsal, open its existing completed run rather than submitting another.

For a new authorized class, **Review → Submit** once. Keep the actual evaluation ID and run ID. Wait for completion and account for all 12 cases, including errors or missing results. A configured draft, HTTP creation receipt or successful model smoke is not a completed evaluation.

Review before Submit. This picture illustrates the controls, not the current Sol run. Confirm the actual pinned version and saved evaluation contract. View full-size image (file: `web/assets/portal/en/17-evaluation-review.png`)

Open **Evaluations → your evaluation name → Evaluation runs → baseline run**. If Running, refresh that run and wait. If it remains unfinished after 30 minutes, record the state/error and notify the facilitator. Stopping your wait does not cancel the remote job.

Use this **read-only command** to retrieve actual evaluation and run IDs. If you chose a different name, use that exact name with `--name`.

```
python -m lab --config .lab/lab-en/.env native-evals --name lab-en-learning-loop
```

Record `evaluation_id` and the `run_id` whose `agent_version` is "1" and status is completed. Evaluation IDs use `eval_...`; run IDs use `evalrun_...`. They are not interchangeable. If several evaluations have the same name, compare their portal creation times, Agents, and runs to select yours. This command submits nothing.

Completion signal: The real Foundry run is Completed and exposes 12 output items. Record failed/error counts in `result_counts` too. A failed or incomplete run stays failed/incomplete. No local custom Judge substitutes for this managed Evaluation.

07. Read scores and reasons

Read summary counts and **Detailed metrics result**, especially `Relevance.reason` and `TaskAdherence.reason`. `conversation_id` → User view opens the actual question/answer, not an inline Judge-reason panel.

Select a failing or lowest-score case and a good case. Connect **question → actual response → score/reason → policy and parsed reference answer**. If no failure exists, say so; never weaken the baseline to produce one.

Microsoft Foundry / proj-fev11-dea3cec5

Q Search with AI (🔍K)

New Foundry Home Discover Build Operate Manage Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

← contoso-eval-en

Make recurring Delete

Run details

Status

Completed

Create time

10/1/2026, 12:21:37 AM

Duration

3m 59.84s

Created by

Agent ID

See all properties

User logs

Results

Download user logs

Download results

Overall metric results

P50 latency	P95 latency	TaskAdherence	Relevance
5.9s	8.8s	100% 12 / 12	83% 10 / 12

Detailed metrics result

Analyze Results

Index	query	conversation_id	trace_id	Relevance
0	Next month's cost is a concern. Please reduce my subscription. I...	conv_07e180556bd61ca800UPWqFCcJqavhmEPmHcTJBXGDhbqHYFFz	-	Fail: 3
1	My first monthly payment was on August 28, 2026 at 08:00 KST. I...	conv_0bb9168bd3bf11ec00jyPvdLhJy3iE1iHfKA6uCVJS9YccWpLx	-	Pass: 5
2	I used the service for just one day after my first monthly payment on...	conv_05f494a8836c0111004DizSunhTEd8q7cEwzgpP3f4M5UKYcT	-	Pass: 4
3	I typed only a title on the Support > Contact support screen and left...	conv_09e82e7f1e3db55a00h7GlpALJkG8N9Ngm0jFoOUAXyEm0k5r	-	Pass: 4
4	There is an unfamiliar access event on the administrator's audit screen...	conv_05740a8abb5d88ad00G5SaTweilR5t4uMTWWidYBV0kq4jgQVM	-	Pass: 5
5	Assume I paid the same monthly base fee in two months, with...	conv_098f970e5fa2d1da00zhmGbIVSMhdOFzI7lUcjb4PGVLH8UFJ	-	Pass: 5
6	I'm on paid Pro, but only beta webhook delivery failed; the...	conv_099677dc019f239900fUJqcz6Qpnwkk97SDxpf0SUV8pXxqrUQ	-	Pass: 5
7	Since there is a 99.9% monthly commitment, can I write in a...	conv_0a857d1c7aec9b6800pluwLFZNNSVYwbrpuCehFCbQhPFYecEUo	-	Pass: 5
8	I learned that the refund window includes the first business day after...	conv_0eb63b9bf392360b00Nh20XhDm9lF7uaitmC44eIDGVB7NA	-	Pass: 5

Find scores and reasons.

This earlier UI image is not current verification. Read the current run IDs and full case evidence in the latest report. View full-size image (file: web/assets/portal/en/18-evaluation-results.png)

Relevance 4/5 is not 80% accuracy. TaskAdherence 1 means Pass, not a low five-point score. Missing values are not zeros or successful rows. Generic evaluators do not certify every business rule.

conv_07e180556bd61ca...

Conversation ID: conv_07e180556bd61c...

Trajectories

User view

Graph view

3 spans1 chat call1.8s1.1K

Next month's cost is a concern. Please reduce my subscription. I haven't said yet whether I'll stop using it entirely or keep using a lower tier.

("answer": "I understand your concern about the upcoming cost. You can reduce your subscription to a lower tier or make other changes to your subscription plan to better fit your needs. Please let me know if you'd like help with specific options or if you want assistance with pausing or canceling your subscription entirely. I'm here to support you in managing your subscription in the best way possible."; "citations": [], "route": "answer", "needs_human": false)

Read the response separately.

Return to Detailed metrics result for the reasons and compare the source policy. Do not copy this older answer as current evidence. View full-size image (file: web/assets/portal/en/19-evaluation-case.png)

Watch for wrong date boundaries, unnecessary assumptions, numeric retrieval IDs instead of document citations, incorrect human routing, invented actions and unsupported certainty. A truthful statement of uncertainty must not be replaced with a fabricated fact to satisfy a Judge.

Create `.lab/lab-en/notes.md` in your editor and fill in this worksheet with your own results. Retain full responses and reasons separately; use the worksheet for identifiable runs/cases and concise observations.

Record	What to write
Baseline	Actual evaluation ID, run ID, Agent version, and dataset hash
Results	Per-metric pass/fail/error counts across all 12 cases
Problem case	Question, problematic sentence in the actual response, evaluator reason, and policy ID
Improvement hypothesis	For a date-boundary mistake, require checking effective dates and inclusive boundaries; do not embed the answer
Behaviors to preserve	Correct citations, honest uncertainty, and action boundaries already working well

Discuss: State the run ID, each metric's scale/pass count, a concrete problematic answer and the instruction behavior you want to improve.

Completion signal: You can explain a real weakness and a supported improvement hypothesis, not just an average.

08. Improve instructions with Agent Optimizer

Open **Build → Agents → lab-en-iq → Optimize Preview/Optimize → Agent**, not Cost. For a new job, select **Create an optimization run** or **Create optimization run**. If preview access or model support differs in your subscription, see [Optimizer troubleshooting](#).

Setting	Choice
Agent version	Explicit baseline 1
Choose targets	Instruction only ; Model and Tool description off
Max candidates	Operator-approved bound; this run uses at most 2 , not more releases
Optimization model	Your <code>OPTIMIZER_DEPLOYMENT</code> / gpt-5.5
Evaluation model	Your <code>JUDGE_DEPLOYMENT</code> / gpt-6-luna
Dataset	Same registered English dev12 version 1
Criteria	Relevance 4; TaskAdherence binary pass 1

1

Target

Optimization tunes your agent's instructions and tool descriptions. Optimization can also tune your agent's quality by attempting different models. To also compare models, turn on Compare across models and pick which ones to include.

Goal
Improve quality
Raise evaluation quality above your current version, with token cost left free.

Agent

Version

contoso-eval-en

v1 (latest)

Optimization approach

☐ Auto

Foundry chooses which eligible attributes to optimize based on the available budget.

☒ Choose targets

Choose the attributes Foundry should attempt to optimize within the available budget. Targets that do not apply to your agent are skipped.

☒ Instruction

☐ Tool description

☐ Model

Optimization model

lab-planner-dea3cec5

Max candidates

1

Evaluation model

lab-judge-luna-dea3cec5

Next

2

Data

3

Review

How optimization works

Scored and ranked

Each candidate is scored against your criteria and ranked by performance.

Verify before applying

Review the top candidate before applying changes to your agent.

Separate the model roles. Use the current operator-provided Sol target, Luna Judge and supported generator; this capture only locates the controls. View full-size image (file: `web/assets/portal/en/07-optimizer-target.png`)

If Criteria shows **No custom evaluators available**, switch **Custom only OFF** or select **View built-in evaluators**. Open each built-in row, set its correct threshold and Apply. Do not create a custom scorer to bypass a filter.

Create an optimization run

Target

Dataset

Criteria

Review

Choose an existing dataset to test each candidate against. Optimization runs the same dataset across every candidate so results are comparable.

Generate data

Select dataset and criteria

Choose an existing dataset, then configure evaluation criteria.

Back

Next

Select dataset

Search by file name

Create dataset from traces

Upload dataset

Name	Version	Created by	Created on
eval-data-2026-09-30_144335_a656c_UTC	1		9/30/26, 11:43:35 PM
contoso-eval-en-dev12	1		9/30/26, 11:20:29 PM

1-2 of 2

PrevNext

Preview

Dataset

contoso-eval-en-dev12

Fields

contextground_truthquery

Dataset preview (Top 5 rows)

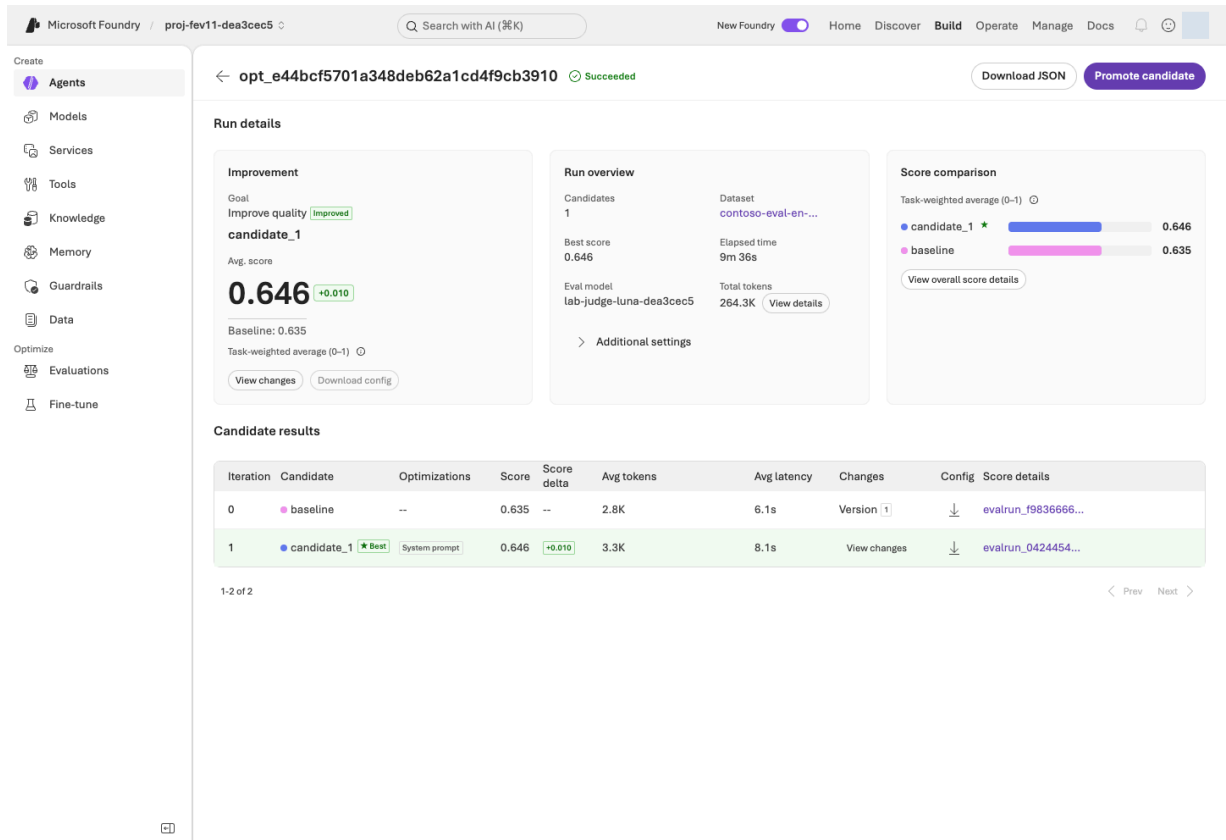
context	ground_truth	query
[ATLAS-SUB-001] Subscription-3. Canceling a subscription stops the next automatic renewal; access continues until the end timestamp of the current	{"answer": "By 'reduce', do you m stopping automatic renewal, or on a lower plan from the next rer Those are different choices, so I clarify your intention first. This c has not changed the	Next month's cost is a concern. Please reduce my subscription. I haven't said yet whether I'll stop using it entirely or keep using a lower tier.
[ATLAS-REF-001] Refund-2. A first purchase of a prepaid monthly subscription made before 2026-09-01 00:00:00 KST retains the previous 7 x 24-hour application window,	{"answer": "The instruction in the notice does not change the polici 28 purchase retains the previou window, whose final timestamp September 4 at 08:00 KST. Septe past the deadline; the month of	My first monthly payment was on August 28, 2026 at 08:00 KST. I haven't used it, and I'm contacting you on September 6. The notice I attached says 'Everyone
[ATLAS-REF-001] Refund-1. A first purchase of a prepaid monthly subscription made at or after 2026-09-01	{"answer": "The standard conditi both no paid-credit use and no p production jobs. If even one paic job was run, the situation you de	I used the service for just one day after my first monthly payment on September 8, 2026. Paid-credit usage is

Reuse the data. Same registered version and all 12 cases; synthetic regeneration or a changed upload would change the experiment. View full-size image (file: `web/assets/portal/en/08-optimizer-dataset.png`)

In **Review**, confirm the Agent/version, dataset, evaluators, and estimated cost, then **Submit** once within the approved scope. An estimate is not a billing cap. Open the job in **Optimization runs**, wait at most 60 minutes, and retain its actual state. Reuse the same job ID when resuming. Optimization includes multiple internal calls.

Inspect original/candidate scores and **View changes**. The internal **0–1 ranking** is not the separate managed Evaluation mean or pass percentage. Keep model, tools, reasoning and output schema unchanged; an empty function-tool export does not authorize removal of the MCP policy connection.

Current run: the managed Optimizer retained the strong v1 baseline. Its generated candidates were not automatically promoted. The operator used the observed failures to refine the instructions and verified that reviewed draft separately. The current v2 source is therefore **operator-reviewed after Agent Optimizer**, not the service's automatically recommended candidate.



Read the candidate, not just the ranking. This image is an older UI example. The current report identifies the actual job and selected candidate. View full-size image (file: [web/assets/portal/en/09-optimizer-results.png](#))



Review the actual current diff. Reject invented policy, unsupported certainty or configuration changes. Longer instructions are not automatically better. View full-size image (file: [web/assets/portal/en/10-optimizer-changes.png](#))

Bring the candidate into the next step. Select it in the result and copy the complete revised instructions from **View changes**. Save them as UTF-8 plain text in `.lab/lab-en/candidate.txt`. Do not include diff `+/-` markers, UI explanations, or scores. If you edit the instructions, record "operator-reviewed after Agent Optimizer" and the reasons in `notes.md`. The repository's `prompts/en/candidate.txt` is an authored example, not the output of your new service run.

Do not continually create v3, v4 and later releases. This guide uses the next step's CLI to create one v2 for comparison, so **do not use both Promote candidate and the CLI creation command**. Experienced operators can validate explicit drafts before promotion, but that is not a hidden prerequisite for this path. Creating a version is separate from publishing it or approving activation.

Discuss: Which instruction behavior changed, what should improve, and what could regress? Identify the actual candidate rather than copying an old screenshot.

Completion signal: Record real job/candidate IDs, the reviewed instruction file, and change reasons. If no candidate is worth retaining, record "keep v1; no improvement observed" and continue at 10. Do not repeatedly run the same job to manufacture improvement.

09. Reevaluate and compare v1/v2

Goal: Evaluate changed instructions while keeping the model, tools, dataset, evaluators, and Judge fixed. First create v2 of the same Agent once from your reviewed file. This preserves v1 and does not publish to production.

```
python -m lab --config .lab/lab-en/.env native-agent --version 2 --prompt .lab/lab-en/candidate.txt --confirm
```

Confirm `agent_name: lab-en-iq` and `version: "2"`. If a different v2 already exists or the model deployment changed, stop and preserve the original records. A new version alone does not demonstrate improvement.

Use the **same Foundry evaluation definition**. The portal's Add run previously failed with `Unable to create data source configuration from item schema`. This official Azure AI Projects/OpenAI Evals helper adds a real managed run to the existing definition; it is not local scoring.

Placeholder	Where to find your value
YOUR_PROJECT_ENDPOINT	AZURE_AI_PROJECT_ENDPOINT in <code>.lab/lab-en/.env</code>
YOUR_SUBSCRIPTION_ID	The subscription ID from 01, also AZURE_SUBSCRIPTION_ID in that <code>.env</code>

Placeholder	Where to find your value
YOUR_EVALUATION_ID	The selected <code>evaluation_id</code> from the <code>native-evals</code> output in 06
YOUR_BASELINE_RUN_ID	That evaluation's completed version-1 <code>run_id</code> , not an Optimizer job ID

Replace all four values and execute this single-line command. Submitting a new run incurs charges.

```
python scripts/add_foundry_eval_run.py --endpoint "YOUR_PROJECT_ENDPOINT" --subscription "YOUR_SUBSCRIPTION_ID" --evaluation "YOUR_EVALUATION_ID" --baseline "YOUR_BASELINE_RUN_ID" --version 2 --name candidate-v2 --out .lab/lab-en/artifacts/foundry-evaluations/candidate-v2.json
```

Confirm status: `completed` and all 12 cases. If the default 30-minute wait expires, the command returns exit code 2 with `Still running`. If the receipt contains a run ID, repeat the **identical command and** `--out path` to collect that run. For an unknown submission without a run ID or an existing remote name, follow [duplicate-submission recovery](#). Do not delete the receipt or rename the run to resubmit.

The helper verifies the remote thresholds, Judge, mappings, and every output item's actual version and system instructions. Historical rehearsal responses and scores cannot substitute for your new run.

In **Evaluation runs**, select both rows and **Compare runs**. Explicitly choose **v1 as Baseline**; selection order must not reverse the comparison.

Microsoft Foundry / proj-fev11-dea3cec5

Search with AI (🔍K)

New Foundry Home Discover Build Operate Manage Docs

Create

- Agents
- Models
- Services
- Tools
- Knowledge
- Memory
- Guardrails
- Data

Optimize

- Evaluations
- Fine-tune

← Statistically compare results

Total runs selected 2 Analysis method PairedTTest

Baseline * ⓘ
contoso-eval-en

Name	contoso-eval-en	candidate-v2
TaskAdherence	1.00 Baseline	1.00 Inconclusive
Relevance	4.42 Baseline	4.33 Inconclusive

1-2 of 2

Prev Next

Confirm the comparison direction. This capture locates the Baseline control; it is not the current Sol measurement. Use the latest report's exact two run IDs. View full-size image (file: [web/assets/portal/en/20-evaluation-comparison.png](#))

The current [v2 verification](#) publishes all 12 case responses, scores and reasons alongside the v1 control. Require no decline in all-criteria passes or either metric's pass count/mean, and at least one strict measured improvement. Check factual truth, routing and response format too. Report latency, tokens and the native statistical conclusion separately.

Explain the result: Identify the actual gain, unchanged criteria, any regression, and the remaining uncertainty. Observed improvement is not a guarantee that every future stochastic run will improve.

In `notes.md`, record v1/v2 pass counts and means per metric, errors, latency/tokens, statistical results, and your decision side by side. If improvement criteria are not met, **retain v1 or hold the decision**. Do not adopt a candidate with actual policy errors merely because generic evaluators passed it.

Completion signal: Record the two actual run IDs, complete same-criteria comparison, and the reasons for accepting, retaining, or holding. Do not accumulate release numbers. Production approval and independent generalization are separate and are not granted by this dev12 exercise.

10. Save results and delete resources

Goal: Preserve necessary evidence and remove unused lab resources. **Closing a browser, deleting an Agent, or running `cleanup` does not by itself stop all resource-group charges.**

Preserve records before deletion

1. Record the actual project, Agent, dataset version/hash, evaluation/run/job IDs, and decision in `notes.md`.
2. Use **Download/Export** where offered in the evaluation and Optimizer views. If unavailable, retain existing receipts and detailed results; do not claim an export you could not obtain.
3. Retain `.lab/lab-en/` plans, manifest, approvals, and run records in an organization-approved private location. Do not commit raw credentials, cookies, signed URLs, or account details. Public records should contain only the necessary synthetic cases, scores, and reasons.
4. Inspect Evaluations and Optimization runs for active jobs. Use **Cancel** where supported and verify the terminal state. Hand unresolved active jobs to the cleanup owner.

Distinguish dedicated and shared environments

Environment	Cleanup scope
Dedicated resource group created for you in 02	Follow the group-deletion procedure below. Reconfirm that no unrelated resources are present and deletion is authorized.
Operator-provided shared project	Do not delete the resource group, Foundry resource, shared models, or Search service. Remove only assigned objects and hand remaining resources to the operator.
Setup or execution stopped partway through	Compare <code>config.json</code> , the manifest, and actual Azure resources. Failure does not mean nothing was created.

For shared resources with local ownership records, first inspect this **plan-only** command:

```
python -m lab --config .lab/lab-en/.env cleanup
```

Read mode: `LOCAL_PLAN_ONLY`, `actions`, `never_deleted`, and `manual_follow_up`. Verify names, project, and ownership. Run the following only after obtaining authorization for exactly those deletions. If your environment name differs, use its matching prefix.

```
python -m lab --config .lab/lab-en/.env cleanup --confirm-prefix lab-en
```

Confirm `OWNED_OBJECTS_ABSENT`. This removes recorded Agent versions, search objects, and connections, but **not resource groups, model deployments, Search hosting, logs, or RBAC**. It does not automatically remove every portal-created dataset, evaluation, Optimizer job, Playground conversation, or model-smoke response. Delete authorized items individually in their owning view; if no delete action is offered, hand retention or project cleanup to the operator.

Delete a dedicated group and verify its absence

Replace `YOUR_LAB_RESOURCE_GROUP` with `names.resource_group` from `config.json`. Verify subscription, ownership tags, and the entire inventory, not just a familiar prefix.

```
az group show --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP" --query "{name:name,location:location,tags:tags}" -o json
az resource list --subscription "YOUR_SUBSCRIPTION_ID" --resource-group "YOUR_LAB_RESOURCE_GROUP" --query "[].{name:name,type:type}" -o table
```

Resource-group deletion is irreversible and removes the group's Foundry resources, model deployments, Search, and monitoring together. After saving evidence and receiving approval for that exact group, choose one method:

1. **Portal:** Azure Portal → Resource groups → exact group → **Delete resource group**. Read the deletion inventory, type the requested group name, and confirm.
2. **CLI:** Run the command below and review the target again at the confirmation prompt. Do not append `--yes` to bypass confirmation.

```
az group delete --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP"
```

An accepted request is not completed deletion. Check portal notifications and group status, then verify that this command successfully prints `false`:

```
az group exists --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP"
```

Authentication or network errors are not `false` and do not prove deletion. If `true`, inspect deletion progress. For **Locks**, permissions, or dependencies in other groups, follow [deletion troubleshooting](#). Do not remove organizational locks without authorization.

Afterward, open **Cost Management → Cost analysis** for the subscription, time range, and group. Billing updates can lag, and earlier usage charges do not disappear. Separately check logs/storage in other groups and service-specific soft-deleted resources. Permanent deletion/purge requires organizational policy and separate authorization.

Final completion signal: Verify the dedicated group's absence and record the time, or hand over a shared-resource inventory with reasons, owner, and retention deadline. Do not delete `.lab` first and lose ownership evidence. See the [operator cleanup worksheet](#) and [execution issue record](#) for the remaining boundaries.

Facilitator guide • improve the measured result

[Ten participant steps](#) • [Operator handoff](#) • [Execution issues](#) • [Latest v2 verification](#)

Teach **evaluate** → **learn** → **improve** → **reevaluate** using business-specific tasks and real managed results. Do not manufacture a weak baseline or promise that every future evaluation will improve.

Before class

Decide whether participants perform 01–03 or receive an operator-prepared environment. Self-setup requires access, cost authorization, and a cleanup owner. Prepared classes need a complete [handoff](#) and verified project, Agent, retrieval, and dataset registration.

New lab Agents use `lab-en-iq` or `lab-ko-iq`. Do not assume the historical `contoso-eval-en-sol` already exists in a new project. Verify [model roles and identity](#), and reserve time for cleanup in 10.

Both v1 and v2 are full immutable Agent versions. Their model, tools, reasoning, and strict output schema match; only instructions differ. Do not weaken v1 to create improvement. The default path creates one reviewed v2 and reevaluates it; only operators who verified draft support use a separate draft workflow.

Keep English and Korean data/Agents separate. The Korean guide explains the procedure; the current published measurements are **English-data results only**.

Verify completion across all ten steps

Step	Required action	Completion evidence
01 Environment	Check account, subscription, permissions, Python, CLI, and language variables	Matching identity and successful local commands
02 Provisioning	Plan, authorize, preflight, apply, and inspect status	APPLIED, generated <code>.env</code> , and runtime preflight PASS
03 Agent setup	Verify model/retrieval and create <code>native-agent v1</code>	Actual JSON response and Agent tool call
04 Dataset	Register or reuse unchanged three-column JSONL	All 12 rows, registration/version, and SHA-256

Step	Required action	Completion evidence
05 Criteria	Relevance 4, TaskAdherence 1, and the actual Judge	Saved remote definition and query-only input
06 Baseline	Foundry Evaluation against pinned v1	Completed actual run ID and all 12 items
07 Analysis	Read actual answers, scores, reasons, and policies	Concrete improvement hypothesis, including failures
08 Optimization	Instruction-only optimization and candidate-diff review	Job/candidate IDs, reviewed file, and provenance
09 Reevaluation	Evaluate same-criteria v2 and open Compare runs	Complete paired evidence and retain/accept/hold decision
10 Cleanup	Preserve evidence, inspect scope, authorize, and delete	Dedicated-group absence or a shared-resource retention handoff

Use **Individual turns / One time / Existing dataset**. The upload preview can show only five rows; the experiment still contains 12. Version selection can clear the target checkbox: reselect it and confirm one target.

Discuss the actual cases

Ask participants to connect **question → actual answer → evaluator score and reason → policy/reference answer**. `conversation_id` → User view shows the conversation; the evaluator reasons belong in **Detailed metrics result**, such as `Relevance.reason` and `TaskAdherence.reason`.

Discuss at least one weak case and one strong case. If no failure is observed, use the lowest-scoring case and say no failure was found. Never weaken v1, omit a difficult row, change a reference answer or substitute local scores.

The latest public record includes all 12 synthetic questions, both actual responses and the managed evaluator reasons. Those results are not credentials. Exclude tokens, cookies, signed URLs and private account metadata, not unsuccessful outcomes.

Interpret improvements correctly

Relevance 4/5 is not 80% accuracy. TaskAdherence 1 is Pass, not a poor one-out-of-five score. Report each evaluator separately, all-criteria passes, errors and coverage.

`scripts/compare_foundry_eval.py` requires complete matched cases, identical data/model/tools/settings, per-item version/instruction attestation, no lower pass counts or means, and at least one strict measured quality improvement. Malformed responses remain failures; the helper never repairs them into a pass.

Read answers for unsupported certainty, incorrect policy arithmetic, action claims and wrong routing even if the generic evaluator passed them. A request demanding an impossible certainty must not be satisfied with invented facts.

Optimizer ranking is internal selection evidence, not the direct managed reevaluation mean or pass rate. A higher ranking alone does not establish a better v2.

Recover without changing the experiment

Symptom	Response
Wrong identity, 401/403	Confirm the approved account/project; do not widen shared permissions
Catalog or deployment succeeds but Agent fails	Record the actual API/runtime error; verify the intended role
Portal Add run item-schema error	Use the official SDK helper with the existing baseline data source, not a different scorer
No custom evaluators available	Switch Custom only OFF or select View built-in evaluators
Duplicate local receipt or existing remote run	Retrieve that same run; do not rename it to force another submission
Candidate changes model/tools/reasoning/schema	Reject an instruction-only comparison claim
Draft becomes a numbered release	Stop; draft support is not enabled as expected
Existing released v2 differs from local source	Do not overwrite it or create v3; preserve the release and inspect the mismatch
Missing output or metrics	Report incomplete execution; do not reduce the denominator
Statistical comparison is Inconclusive	Report that result; do not claim significance or equivalence

Relevance maps to `{{sample.output_text}}`; TaskAdherence maps to `{{sample.output_items}}`. Do not overwrite service-generated bindings or feed `ground_truth` into the Agent. The actual remote Judge overrides any stale local default.

After a timeout, identify **which existing job ID to resume** first. Use the [issue record](#) to prevent deletion/recreation or paid retries “until it succeeds.” Do not mark a blocked step completed.

Correct common misconceptions

Claim	Correction
"V2 is a prompt file number only."	It is a complete Agent version; this experiment changes only its instructions
"A more powerful model makes v2 better than old-model v1."	Both versions must use the same model for the instruction comparison
"A better v2 can be guaranteed before execution."	Only observed improvement can be established; future stochastic results are not guaranteed
"Keep only successful rows."	Keep every case, including failures and missing/error outcomes
"The source schema certifies correct business decisions."	It controls output structure, not policy truth or proper routing
"Public evidence must include cookies and raw logs."	Publish actual synthetic responses and reasons through an allowlist, not authentication data

Finish with one current report

Publish **the latest v2 verification and its frozen v1 control**, not a sequence of obsolete current-version reports. Keep earlier raw receipts for audit without relabeling them as this pair.

Report the same-model result, native statistical comparison, any latency/token tradeoffs and the remaining limits of a reused synthetic dev set. Candidate activation is a separate operator decision; no production publication is part of this workshop.

Before ending class, perform the cleanup checks in 10 together. Confirm dedicated-group deletion or a named owner and deadline for retained shared resources. Hand off actual outcomes and issue records privately; do not delete ownership records first.

Operator setup, authorization, and cleanup

[Ten steps from the beginning](#) · [Facilitator](#) · [Execution issues](#) · [Latest v2 verification](#)

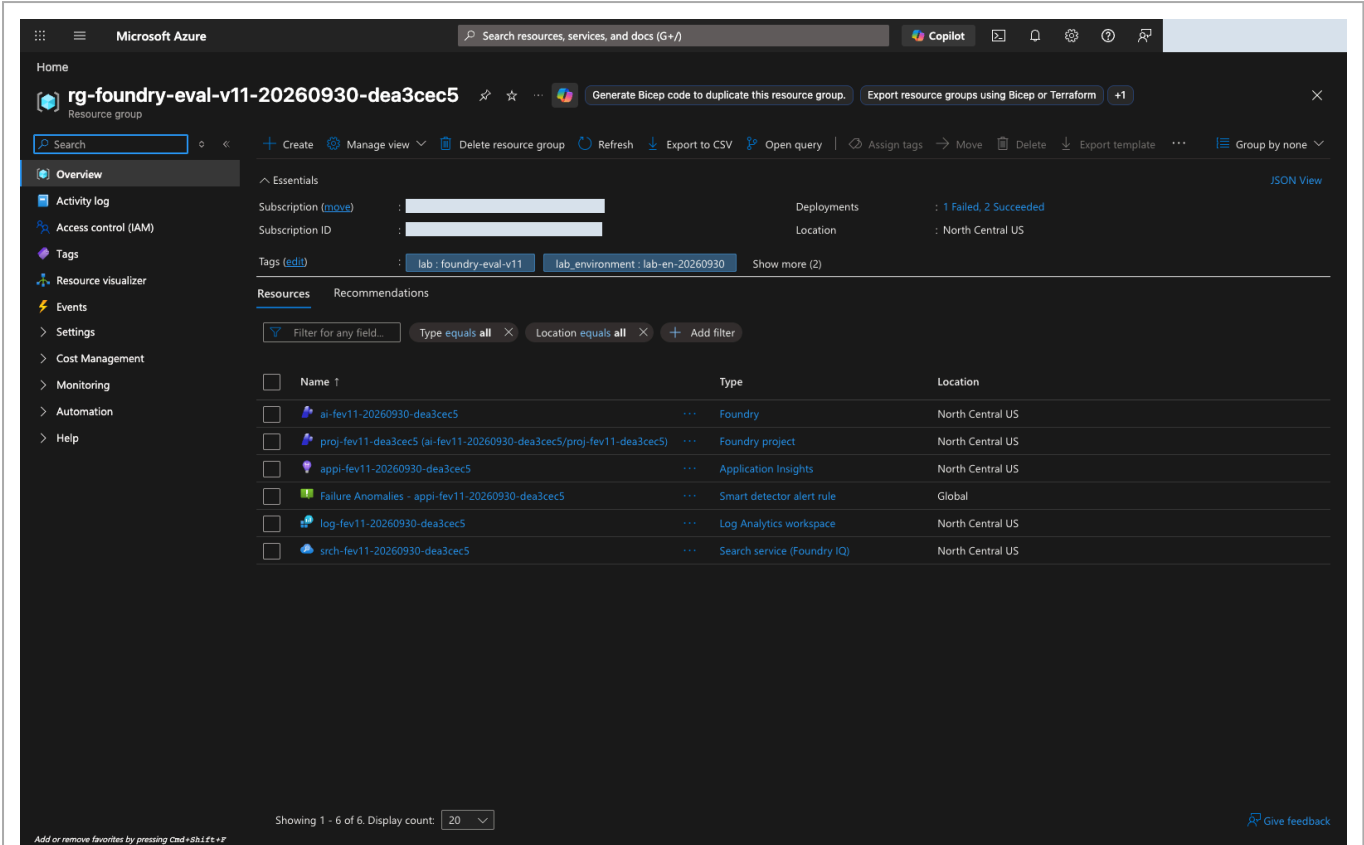
Use this reference for participant steps **02 provisioning authorization, 03 Agent setup, and 10 deletion**. Participants preparing their own environments can use it too. Supply actual values and completion evidence rather than leaving prerequisites implicit.

Distinguish new and prepared environments

Situation	Action
No project exists	Start with 01 account setup and use the authorized bootstrap plan → preflight → apply → status path.
The operator already prepared this lab	Supply actual project, Agent, data, and access through the handoff ; participants do not duplicate creation.
Only an existing shared project is available	Bootstrap does not silently adopt existing groups. Its owner must define the preparation/change scope and approve project-specific access.

Verify CLI, SDK, and portal identities separately. Never transfer the operator's token or sign-in session to a participant. The owning operator runs bootstrap-bound creation commands. Other participants use their own assigned portal roles; SDK access requires separate configuration and permissions matching their own identity.

New examples use `lab-en-iq` for English and `lab-ko-iq` for Korean. Historical `contoso-eval-en-sol` and `lab-...-dea3cec5` deployment names are references, not fixed names to use in new environments.



Resource-group orientation only. This earlier portal picture is not the current v2 result. Verify the actual project and current state from the operator handoff. View full-size image (file: web/assets/portal/en/00-resource-group.png)

Verify model roles and actual deployments

Role	Configuration key	Default model/version
Agent	MODEL_DEPLOYMENT	gpt-6-sol / 2026-09-22, GlobalStandard 20
Managed Evaluation Judge	JUDGE_DEPLOYMENT	gpt-6-luna / 2026-09-22, GlobalStandard 20
Optimizer/search planner	OPTIMIZER_DEPLOYMENT / IQ_PLANNER_DEPLOYMENT	gpt-5.5 / 2026-04-24, one shared GlobalStandard 20 deployment
Policy embeddings	EMBEDDING_DEPLOYMENT	text-embedding-3-small / 1, GlobalStandard 10

Numbers are requested ARM capacity units, not a universal TPM conversion. These defaults require availability checks; they are not a deployment guarantee for every subscription. Inspect deployment names, model names, versions, and readiness in Foundry **Models + endpoints/Build → Models**. .env contains actual deployment names rather than product names.

Historical Sol measurements verified actual Agent and `knowledge_base_retrieve` calls, not just a catalog entry. Repeat the [real response check in 03](#) for a new environment. Optimizer has a separate [supported-model list](#) (<https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models>); Agent/Judge availability does not establish generator support.

Microsoft Foundry / proj-fev11-dea3cec5

Search (🔍)

Create

Agents

Models

Services

Tools

Knowledge

Guardrails

Data

Optimize

Evaluations

Fine-tune

Models

Deployments Batch jobs

Serverless deployments Managed compute deployments

Search

PTU Calculator

Deploy

Name	Model	Version	Deployment status	Deployment type	Created on ↓	Created by
☒ lab-judge-luna-dea3cec5	gpt-6-luna	2026-09-22	Succeeded	Global Standard	9/30/26, 11:19:02 ...	
☐ lab-agent-luna-dea3cec5	gpt-6-luna	2026-09-22	Succeeded	Global Standard	9/30/26, 11:18:59 ...	
☐ lab-embedding-dea3cec5	text-embedding-3...	1	Succeeded	Global Standard	9/30/26, 9:58:45 PM	
☐ lab-planner-dea3cec5	gpt-5.5	2026-04-24	Succeeded	Global Standard	9/30/26, 9:58:44 PM	
☐ lab-judge-dea3cec5	gpt-5.4-mini	2026-03-17	Succeeded	Global Standard	9/30/26, 9:58:42 PM	
☐ lab-agent-dea3cec5	gpt-4.1-mini	2025-04-14	Succeeded	Standard	9/30/26, 9:58:40 PM	

1-6 of 6

Prev Next

Locate model/version fields. The picture predates the Sol deployment. Use the role table and current recorded API observations, not this older list, as runtime evidence. View full-size image (file: `web/assets/portal/en/02-model-deployments.png`)

Prepare an executable Agent

First complete Python installation, virtual-environment creation, login, and language variables using the [copyable commands in 01](#). Do not activate a nonexistent `.venv` or treat fake IDs in `.env.example` as a live environment.

Use `prompts/en/baseline.txt` for English and `prompts/baseline.txt` for Korean. Never weaken the baseline to manufacture improvement. After `iq prepare` → `iq probe` succeeds, create a policy-connected v1:

```
python -m lab --config .lab/lab-en/.env native-agent --version 1 --confirm
```

`native-agent` uses `ensure_fixed_release` to allow only **1 and 2**, comparing local workspace ownership with remote metadata. It reuses matching configurations, rejects a different v2, and never creates v3. It also retains a model-deployment snapshot and rejects drift. Receipts `agents/native-v1.json`, `agents/native-v2.json`, and `agents/native-model.json` belong to that environment's artifacts folder.

`native_response_format()` projects the common schema into the supported strict-generation subset. Citation uniqueness and `needs_human/route` consistency still require post-generation checks. Valid JSON does not establish policy correctness.

Creating v2 is not completing evaluation. The default path creates one unpublished v2 from reviewed instructions, runs the same managed evaluation, and then decides whether to accept it. Experienced operators may first test `draft-...` versions as in the historical rehearsal, but must verify subscription support. Draft validation is not an unstated prerequisite for the default path.

```
python -m lab --config .lab/lab-en/.env native-agent --version 2 --prompt .lab/lab-en/candidate.txt --confirm
```

Do not combine portal Promote and CLI creation. V1/v2 are complete Agent versions; the same Agent, model, tools, and output settings must differ only in instructions.

Retrieve IDs with `native-evals --name lab-en-learning-loop`. The reevaluation helper copies the baseline data source and checks the **remote** contract. Do not replace the remote definition with a stale local `JUDGE_DEPLOYMENT`.

Freeze the comparison conditions

Item	Fixed requirement
Data	Same selected-language dev12 bytes, registration/version, questions and references
Relevance	Managed <code>builtin.relevance</code> , scale 1–5, threshold 4
TaskAdherence	Managed <code>builtin.task_adherence</code> , binary 0/1, pass 1
Judge	Same explicit Luna deployment in both evaluator definitions
Agent	Same Sol model/version, tools, reasoning and strict output schema
Change	Instructions only; releases remain v1/v2
Optimizer	Instruction target only, model/tool-description changes off, bounded candidate count

Item	Fixed requirement
Publication	Latest v2 and its frozen v1 control; all 12 sanitized case outcomes

Catalog evaluator versions are not proof that the private service rubric is fully pinned. Preserve the actual definition, settings and this limitation. A passing gate on reused dev12 does not guarantee future scores, independent generalization or production approval.

Complete the actual authorization file

Open `approval.example.json` and **Save As `approval.json`**. This worksheet explains fields; it is not an approved request. Keep the actual organizational authorization private. Use double quotes for JSON strings, but not for numbers, true, false, or null.

Field	How to complete it
<code>schema_version</code> , <code>scope_sha256</code> , <code>models</code> , <code>retention_days</code>	Preserve generated values. Editing the model array or plan invalidates existing authorization.
<code>approved</code>	Set <code>true</code> only after actual approval.
<code>approved_by</code>	The operator sign-in name matching <code>expected_user</code> in <code>config.json</code> .
<code>approved_at</code> , <code>expires_at</code>	Actual authorization start/end in timezone-aware ISO 8601, such as <code>2026-10-03T09:00:00+09:00</code> . Do not reuse the example date. The current time must fall within the interval.
<code>currency</code> , <code>budget_amount</code> , <code>budget_policy</code>	Approved three-letter currency and positive numeric budget. Keep policy <code>BOUNDED</code> ; the budget owner determines the amount.
<code>acknowledge_no_monetary_cap</code>	Keep <code>false</code> . Unlimited spending is not the default lab policy.
<code>max_hosting_hours</code>	Approved positive integer hours. If 8, the approval interval must also be no longer than 8 hours.
<code>max_wait_seconds</code>	Positive integer wait limit, for example 3600. Ending the wait does not cancel a remote job.
<code>max_calls</code> , <code>max_candidates</code>	Approved call/candidate limits. The candidate example is 2; consider internal Agent, Judge, and search calls when setting the call limit.
<code>allow_global_inference</code>	<code>true</code> only when GlobalStandard processing is authorized; it is not NCUS-only processing.

Field	How to complete it
allow_resource_creation, allow_rbac_assignments	Each is <code>true</code> only when this plan's resources and resource-scoped role assignments are authorized.
acknowledge_continuous_hosting, acknowledge_unknown_cost	Each is <code>true</code> only after acknowledging continuing hosting charges and an unconfirmed final cost.
allow_training, allow_global_training	<code>false</code> for this path.
max_epochs, max_training_jobs	<code>0</code> for this path.
accept_deprecated_models	Default <code>false</code> ; do not change it without reviewing a service deprecation notice.

Validate with `preflight --approval ...` and read `approval_reason` on failure. Authorization covers only the exact plan hash, models, and retention scope. It does not authorize another class, an automatic retry, or deletion.

Enforcement boundary: Bootstrap checks authorization validity, provisioning scope, and its wait bound. JSON budget and call limits do not automatically cap all Azure Portal or Optimizer spending. Budget alerts are not cutoffs. The operator monitors actual usage and cancels jobs or removes resources when required.

Verify permissions and provider registration

Follow the [current Foundry RBAC guidance \(https://learn.microsoft.com/azure/foundry/concepts/rbac-foundry\)](https://learn.microsoft.com/azure/foundry/concepts/rbac-foundry). **Foundry User/Owner/Account Owner/Project Manager** may still appear under their former **Azure AI User/Owner/Account Owner/Project Manager** names.

Identity or task	Required scope to verify
New-environment operator	This bootstrap checks effective subscription permissions for group/deployment/resource writes and <code>Microsoft.Authorization/roleAssignments/write</code> . An already-authorized provisioning operator performs it.
Portal participant	Agent/evaluation operations in the assigned project; an administrator reviews the required project-scoped roles, such as Foundry User.
Policy-setup operator	Search schema/upload permissions, such as Search Service Contributor and Search Index Data Contributor, on this Search service.
Project managed identity	Search Index Data Reader on this Search service, model invocation, and telemetry submission to the assigned monitoring resource.

Identity or task	Required scope to verify
Search managed identity	Model invocation on the assigned Foundry resource.
Log reader	Read access to the assigned Application Insights/Log Analytics resources, not subscription-wide log access by default.

In Azure Portal → **Subscriptions** → **selected subscription** → **Resource providers**, verify **Registered** for `Microsoft.CognitiveServices`, `Microsoft.Search`, `Microsoft.OperationalInsights`, and `Microsoft.Insights`. An authorized subscription administrator registers missing providers through organizational procedures. Bootstrap never auto-registers providers or expands permissions.

Allow registration/role propagation, then rerun the same preflight. For continuing 403, compare identity, role scope, and network access. Do not grant new subscription-wide Owner, disable network or organizational policies, or transfer tokens as a shortcut.

Costs and operational boundaries

Evaluation invokes the Agent and Judge; optimization makes additional internal calls. Search and monitoring can continue to incur charges after the lab page closes. Distinguish measured tokens, cost estimates, and actual billing; unknown billing is not zero.

In Azure Portal → **Cost Management** → **Cost analysis**, select the subscription, resource group, and date range. Configure organizational budget alerts where needed, but do not present them as enforced caps. Record the cost/cleanup owner and end time in the handoff.

Hand off actual values without gaps

Copy this table into the private class record and fill it with actual values. “The environment is ready” is not a sufficient handoff.

Required item	Actual values or evidence
Identity	User, tenant ID, and subscription ID; never passwords or tokens
Responsibilities	Who performs provisioning, SDK operations, portal exercises, cost monitoring, and cleanup
Local paths	Environment name, bootstrap config, runtime <code>.env</code> , <code>LAB_LANGUAGE</code> , and <code>LAB_ARTIFACTS_DIR</code>

Required item	Actual values or evidence
Azure environment	Resource group, Foundry account/project, project endpoint, and Search service
Models	Actual names, product/version, SKU, and capacity for all four deployments
Policy search	Source/language, eight uploaded documents, knowledge-base/connection names, and <code>retrieval_verified</code>
Agent	Actual name, pinned v1, strict output, and a successful real Agent tool call
Dataset	JSONL path, 12 rows, SHA-256, registration name, and version
Evaluation	Relevance 4, TaskAdherence 1, Judge deployment, and query-only input; add actual evaluation/v1-run IDs after the first run
Authorization	Scope hash, expiry, budget, candidate limit, and retention
Finish	Exact deletable group/objects, deletion approver, planned time, and shared-resource retention

Historical results are separate in the [verification record](#). Keep English and Korean source data separate; do not relabel English measurements as a Korean execution.

Raw service files can contain account metadata, tokens or signed URLs. Keep those local; **the synthetic evaluation results themselves are not secret**. Publish only allowlisted fields, including failures, and exclude credentials—not inconvenient outcomes. Historical attempts stay in the private audit, not as additional current-version reports.

Verify deletion and retention

Use [step 10](#) as the exit checklist. Provisioning approval is not deletion approval, and workshop documentation does not authorize deleting another person's resources.

Target	Deletion or handoff check
Active evaluation/Optimizer jobs	Cancellation availability, actual terminal state, and unresolved job IDs
Locally owned objects	Exact <code>cleanup</code> plan and verified absence afterward
Portal-created objects	Individually inspect datasets, evaluations, conversations, Optimizer records, and empty Agent entries

Target	Deletion or handoff check
Dedicated resource group	Review all inventory, obtain authorization, delete, and verify a successful <code>az group exists</code> returns <code>false</code>
Shared/external resources	Retained item, reason, owner, and retention end date
Billing/soft delete	Delayed charges, prior usage, and separate service retention/purge policy
Evidence	Keep outcomes and failures privately for the required period; do not delete <code>lab</code> first

Execution issues and resolutions

[Guide from the beginning](#) · [Operator authorization and handoff](#) · [Actual quality measurements](#)

Start with the blocked step, then follow symptom, cause, resolution, and verification. Historical Azure measurements, this revision's read-only observations, and local checks are distinct. Documenting a resolution does not establish that a new cloud execution succeeded.

Scope and how to use this record

Evidence category	Source and limitation
Earlier service execution	The rehearsal recorded by the existing guides, portal captures, and <code>evidence/latest.json</code> , as of 2026-10-01. Published quality measurements use English data.
Read-only observations on 2026-10-03	The existing lab's CLI user, tenant, and subscription matched. <code>native-evals</code> retrieved v1, v2, and a draft run from that same evaluation.
Documentation/code changes on 2026-10-03	Added first-time setup, native Agent creation commands, ID lookup, cleanup boundaries, and formal Korean prose. The final section describes verification scope.
Not performed in this revision	New Azure resource creation, role changes, data uploads, paid evaluations, Optimizer reruns, and actual cloud deletion. Existing measurements are not relabeled as a new Korean run.

For a new issue, record **time, step, command, error code, target ID, change, and recheck result** in private `notes.md`. Before publication, remove subscription/account identifiers, tokens, cookies, and signed URLs. Preserve failed responses and unfavorable evaluation outcomes.

Environment, login, and local files

Symptom	Cause and resolution	Ready-to-continue check
The guide starts with an already-prepared project and Agent	The earlier participant guide had only six evaluation steps; operator setup assumed existing infrastructure. New 01-03 connect account, installation, provisioning, retrieval, and Agent preparation.	Confirm your own project, Agent, and actual tool response; passing link checks is not environment readiness.

Symptom	Cause and resolution	Ready-to-continue check
<code>.venv/bin/activate</code> or <code>Activate.ps1</code> is missing	Create the virtual environment from the repository folder first. Windows and macOS/Linux activation paths differ. Follow the installation sequence .	The venv's <code>python -m pip --version</code> and <code>python -m lab --help</code> succeed.
<code>ModuleNotFoundError</code>	Wrong interpreter or missing locked dependencies. Run <code>python -m pip install -r requirements.lock</code> using the venv Python.	Run dataset checks and the intended command with that same Python.
PowerShell policy blocks activation	Use <code>.\.venv\Scripts\python.exe</code> without changing organizational policy.	The interpreter runs without a policy change.
Portal access works but CLI shows a different account	Browser and CLI authentication are separate. Compare directory/subscription filters and the user, tenant, and subscription from <code>az account show</code> .	All three approved values match; no ambient identity fallback.
<code>AzureCliCredential</code> fails when given both tenant and subscription	The corresponding Azure CLI argument combination is incompatible. <code>lab/auth.py</code> first validates the tenant, then supplies only the subscription to the credential.	The same verified identity/subscription is used; tenant validation remains intact.
Korean runs use English artifacts, or vice versa	Set <code>LAB_LANGUAGE</code> and <code>LAB_ARTIFACTS_DIR</code> before Python starts. Reading <code>.env</code> cannot change an artifacts path already selected at import time.	Language-specific folder and workspace metadata agree; original artifacts remain untouched.

The first two gaps were confirmed by reviewing the previous guide sources. Authentication argument and language constraints are verified implementation/regression-test facts, not newly induced authentication failures or paid calls.

Provisioning, authorization, and quota

Symptom	Action	Verification
Plan returns <code>BLOCKED_AWAITING_APPROVAL</code>	The local plan exists, but spending is not approved. Distinguish <code>plan_status</code> from <code>mutations_performed</code> .	Confirm <code>CREATED_LOCAL_ONLY</code> and <code>false</code> , then complete the approval worksheet .
Approval fails despite <code>approved: true</code>	Possible hash/model mismatch, wrong approver, invalid timezone/date interval, or missing budget/consent. Read <code>approval_reason</code> ; correct only genuinely authorized fields.	Both readiness <code>READY</code> and approval <code>READY_FOR_APPROVED_APPLY</code> are present.

Symptom	Action	Verification
A provider is not Registered	An authorized subscription administrator registers it through organizational procedures. Bootstrap does not register it automatically.	All four providers are registered; rerun the same preflight.
Contributor access fails on role assignments	Resource creation and role-assignment authority are separate. Use an already-authorized provisioning operator.	Effective required access is confirmed; do not grant new subscription-wide Owner as a workaround.
Model/SKU/version/region capacity or quota is blocked	Compare the role-specific plan with preflight reason . If substitution is needed, verify support for each role and obtain a new plan/authorization.	Verify catalog, quota, capacity, and then actual Agent/tool execution.
Apply times out or has an unknown result	Preserve config, manifest, and deployment ID. Inspect bootstrap status and the group's Deployments in Azure Portal.	Wait if active; do not create another environment or resubmit while the outcome remains unresolved.
Environment already exists	This protects an existing plan from overwrite. Use its original config with status.	Resume that lab; only a separate authorized lab receives a new scope.
Old recovery command names do not exist	Actual CLI names are repair-dependencies and repair-trace-routing . The outdated infrastructure documentation names were corrected.	Check <code>python -m lab.bootstrap --help</code> . These are not general retries; matching failure evidence and separate authorization are required.

`--retry` applies only to a verified **terminal failure** of the owned deployment, within an explicit retry allowance. It is not permission to repeat an unknown POST. **APPLIED** establishes infrastructure creation, not model quality, production approval, or successful trace ingestion.

Policy retrieval and Agent setup

Symptom	Action	Verification
A model appears in the catalog but the Agent fails	Deployment success and Agent runtime support differ. Record the real API error, model/version, and tool path.	Complete the smoke and actual Agent call in 03 .
Setup remains <code>created_not_retrieval_tested</code>	Only creation is complete. Run <code>iq probe</code> and inspect response, references, and activity.	<code>retrieval_verified</code> with actual sources.

Symptom	Action	Verification
Direct retrieval succeeds but Agent tool returns 403	Direct retrieval uses the CLI user; the Agent connection uses the project managed identity. Check Search/model permissions and the connection audience for that identity.	A successful <code>knowledge_base_retrieve</code> response inside an Agent execution.
English/Korean search behaves inconsistently	<code>lab/knowledge.py</code> uses the selected <code>en.microsoft</code> or <code>ko.microsoft</code> analyzer. Preserve the original language, index, and document hash.	Distinguish language regression checks from live retrieval; renaming an existing index does not isolate it.
Agent setup is described only by a function name, with no executable entry point	This revision connects <code>native-agent --version 1/2</code> to existing <code>ensure_fixed_release</code> , including policy MCP, strict JSON, ownership, and model snapshots.	Local regression checks cover the command; a fresh environment still requires actual creation and tool-response verification.
Existing v2 has different instructions or the model changed	The fixed-comparison guard stops the operation. Preserve records instead of overwriting v2 or creating v3.	Compare owned Agent, model, tools, output settings, and intended instructions.
<code>PublicNetworkAccess=Disabled</code> or private-network timeout	Use the approved VNet, VPN, or organizational execution environment. Do not disable firewalls/private endpoints.	Have the network owner verify DNS/connectivity, then recheck the same endpoint.

Foundry evaluation and safe resumption

The portal Add run action returned an item-schema error

Previously observed message:

```
Unable to create data source configuration from item schema
```

In that session, portal Add run could not reconstruct the existing Agent-targeted data source. This is not evidence of a service-wide outage or identical behavior in every subscription.

Resolution: Use `scripts/add_foundry_eval_run.py` in 09 to copy the completed baseline's remote data source and change only the explicit candidate version. It submits a real run under the same managed evaluation definition. A local Judge, changed JSONL, or different criteria is not a substitute.

Verification: Confirm evaluation ID, original dataset registration, Relevance 4, TaskAdherence 1, Judge/mappings, and actual version/instructions for all 12 output items. Existing completed v1/v2 runs were also confirmed in this revision's read-only lookup. No new paid run was submitted to reproduce the portal error.

Prevent wrong IDs and duplicate submissions

Symptom	Resolution
Evaluation/run IDs are difficult to find	Use <code>native-evals --name</code> . Select the <code>eval_...</code> evaluation and completed version-1 <code>evalrun_...</code> baseline.
Several evaluations share a name	Compare portal time, Agent, dataset, and runs. Do not choose by name alone.
Helper exits with Still running	If the receipt has a run ID, repeat the same command and output path to collect that run.
Receipt exists without a run ID	Acceptance is unknown. Preserve it and inspect the portal/service. Do not delete the receipt, rename the run, or automatically resubmit.
A remote run already uses that name	Open it or restore its original receipt instead of forcing a duplicate through a new name.
Selecting v1 clears the target checkbox	Reselect the Agent after pinning v1; confirm exactly one target.
Dataset preview has five rows	This can be a preview limit. Compare the source's 12 rows, registered version, and actual result count.
TaskAdherence 1 looks like a low score	It is binary 0/1; 1 means Pass. Do not apply Relevance's 1–5 scale.
Reasons are not shown in the conversation view	<code>conversation_id</code> → <code>User</code> view shows the question/answer. Use Detailed metrics result for <code>Relevance.reason</code> and <code>TaskAdherence.reason</code> .
Output items or metrics are missing	Preserve failures/missing values and the 12-case denominator. Never invent zeros or passes.

Agent Optimizer and result interpretation

Observation or symptom	Resolution and verification
An earlier UI showed No custom evaluators available	Use Custom only OFF or View built-in evaluators . A filter issue does not require creating a different custom evaluator.
Optimize is missing or a model cannot be selected	Check New Foundry, prompt-Agent type, project access, preview availability, and role-specific model support. Record tenant availability rather than claiming success when unavailable.
Candidate changed the model or tool descriptions	Check Choose targets → Instruction only and disable model comparison. Different non-instruction settings invalidate an instruction-only comparison.

Observation or symptom	Resolution and verification
The actual Optimizer retained v1	The execution in <code>evidence/latest.json</code> selected the strong baseline. Do not hide this or relabel it as automatic candidate promotion.
An operator further edited the instructions	Record "operator-reviewed after Agent Optimizer" and perform a separate same-criteria managed evaluation. The published historical v2 follows this pattern.
Optimizer ranking improved but reevaluation differs	Its internal 0–1 ranking is not the separate evaluation mean/pass rate. Review full responses, policy errors, and regressions.
Means improved but statistics are Inconclusive	The historical Relevance result is 4.8333 → 4.9167; service PairedTTest is Inconclusive. This does not establish significance or equivalence.
Latency/tokens increased alongside quality	Historical v2 p95 increased 10.97 → 42.97 seconds; Agent tokens increased 46,167 → 55,858. Report the tradeoff with the quality result.

Original responses, scores, reasons, and actual run IDs are in the [latest quality report](#). Twelve reused synthetic development cases do not establish independent generalization or production safety.

Deletion and ongoing costs

Symptom	Resolution and verification
The earlier guide ended at result handoff	Step 10 now provides dedicated/shared paths, inventory checks, authorization, group deletion, and absence verification.
Search charges remain after cleanup succeeds	<code>cleanup</code> removes owned objects, not Search hosting, models, groups, or logs. Confirm dedicated-group deletion or an operator retention plan.
Missing workspace or <code>.env</code> prevents cleanup	Use the interrupted setup's config/manifest and actual Azure inventory. Do not fabricate local ownership records.
Group deletion fails due to locks, permissions, or dependencies	Inspect Locks , Activity log , and the failed deletion in Azure Portal. Ask the lock owner for scoped action; do not remove it unilaterally.
Deletion is accepted but the group remains	Inspect asynchronous progress. Only a successful <code>az group exists</code> returning <code>false</code> proves group absence.
Cost analysis still shows charges after deletion	Prior usage and aggregation delay can remain. Check date range, resources, and any external retained services.

Symptom	Resolution and verification
Soft-deleted items remain	Follow service-specific retention policy. An authorized owner decides whether separate permanent deletion/purge is necessary.

Follow the [Azure resource-group deletion documentation \(https://learn.microsoft.com/azure/azure-resource-manager/management/delete-resource-group\)](https://learn.microsoft.com/azure/azure-resource-manager/management/delete-resource-group). Group deletion is irreversible; individual service recovery features do not guarantee resource-group recovery.

Reproduction commands and verification scope

Actual read-only observation: On 2026-10-03, the existing approved environment's CLI identity matched all three expected values and its subscription was Enabled. `native-evals` retrieved completed v1/v2 runs with 12 cases each, plus a separate draft run, from `contoso-en-sol-learning-loop`. The v1 11/12 and v2 12/12 counts matched the published report. No new evaluation, optimization, Agent creation, or deletion request was sent.

Local reproduction commands: Run in the virtual environment; these create no Azure resources.

During automated verification, `--help` correctly exited through `SystemExit(0)`, and macOS temporary paths exposed the `/var` symlink. The test harness now recognizes successful help exits and resolves its temporary directory to a physical path. Bootstrap's symlink protection was not weakened, and no user files were moved.

Check performed in this revision	Result
Local execution	Executed both guides' local plans, unapproved-apply refusal, and row-count/hash commands. All 579 automated checks passed.
Browser	Verified both languages at 1440px desktop and 390px mobile. Ten-step navigation, language switching, progress/theme persistence, and JavaScript-disabled navigation worked, with no page-level horizontal overflow or page errors.
Print/PDF	Both complete editions include 12 language-matched portal images. Required content, local-machine links, out-of-page text, and nearly empty-page checks passed.
Platform scope	Actual execution used macOS. Windows PowerShell commands are provided separately, not represented as tested on Windows.

```
python scripts/build_datasets.py --language ko --check
python scripts/build_datasets.py --language en --check
python scripts/build_guide.py --check
python -m unittest discover -s tests -q
```

The first direct PDF-check invocation reported `ModuleNotFoundError: pymupdf`. After reconfirming the import and version in the selected virtual environment, both files passed through the module entry point below. The initial import failure's cause was not established; it is not labeled a defective package. If the PDF dependency is genuinely missing, install `requirements-verification.lock` in the maintenance environment only.

```
python -m scripts.verify_pdf docs/Foundry-Learning-Loop-Lab-K0.pdf --language ko --out
.lab/verification/pdf-ko.json
python -m scripts.verify_pdf docs/Foundry-Learning-Loop-Lab-EN.pdf --language en --out
.lab/verification/pdf-en.json
```

Checks cover the two HTML editions' ten-step sequence, reciprocal anchors, links, command arguments, fixed-Agent ownership/configuration invariants, read-only ID lookup, and duplicate-submission guards. Local mocks do not establish a fresh Azure creation, paid execution, or deletion. A new lab must still satisfy each step's real completion criteria.

Formalizing authored Korean instructions changes the baseline prompt, system messages in supporting generated data, and corresponding manifest hashes. **Policies, questions, reference answers, evaluation dev12, and historical measured output remain unchanged.** Use the original run's instruction snapshot when reproducing an earlier execution; do not overwrite historical hashes with the revised prompt hash.

Official references: [project creation \(https://learn.microsoft.com/azure/foundry/how-to/create-projects\)](https://learn.microsoft.com/azure/foundry/how-to/create-projects), [Foundry RBAC \(https://learn.microsoft.com/azure/foundry/concepts/rbac-foundry\)](https://learn.microsoft.com/azure/foundry/concepts/rbac-foundry), [Agent Optimizer \(https://learn.microsoft.com/azure/foundry/agents/quickstarts/quickstart-optimize-prompt-agent\)](https://learn.microsoft.com/azure/foundry/agents/quickstarts/quickstart-optimize-prompt-agent), and [resource-group deletion \(https://learn.microsoft.com/azure/azure-resource-manager/management/delete-resource-group\)](https://learn.microsoft.com/azure/azure-resource-manager/management/delete-resource-group).

Latest v2 verification · gpt-6-sol

V2 improved the measured quality result over the same-model v1: all-criteria passes increased **11/12 → 12/12**, mean Relevance **4.8333 → 4.9167**, and TaskAdherence remained **12/12**. This page reports only the current Sol pair, not historical experiments.

Important limit: p95 latency increased **10.97 → 42.97 seconds**, and the native statistical comparison is **Inconclusive**. The measured quality gain does not establish production readiness, statistical significance or guaranteed improvement on future runs.

What was measured

Both versions of `contoso-eval-en-sol` used the same `gpt-6-sol / 2026-09-22` Agent, read-only policy tool, reasoning setting, strict JSON schema and registered `contoso-eval-en-dev12` **version 1**. Only the instructions changed.

All 12 unchanged records came from `data/en/optimizer/dev.jsonl` (file: `data/en/optimizer/dev.jsonl`). Only `query` was sent to the Agent; `context` and `ground_truth` remained references. The two managed evaluators and their Luna Judge were unchanged. Actual version and system instructions were checked in every result item.

These are **actual English-data results**, not a newly executed Korean run. Local unit tests and response-format checks do not replace Microsoft Foundry managed Evaluation.

Final v1/v2 results

Metric	Sol v1 control	Sol v2
Cases	12	12
All-criteria passes	11/12	12/12
Relevance passes, threshold 4	11/12	12/12
Mean Relevance, 1–5	4.8333	4.9167
TaskAdherence, binary pass 1	12/12	12/12
Execution errors / skipped cases	0 / 0	0 / 0

Metric	Sol v1 control	Sol v2
Agent p50 latency	8.03 s	9.71 s
Agent p95 latency	10.97 s	42.97 s
Observed Agent tokens	46,167	55,858

Evaluation definition: eval_40a593c037e44045a47fe5088438afc5.

V1 run: evalrun_077d41ff8a534b9caee64f9d6c2a339d. Final released v2 run: evalrun_6c9e78cfc9de4f1282eb621c6678d8b1. The report uses the actual released version **2**, not a draft relabeled as v2.

Quality acceptance: PASS for this measured lab pair. No evaluator pass count or mean declined, one additional case crossed the Relevance threshold, and all candidate responses met the JSON/routing/citation checks. Production approval is **not granted**, especially given the observed latency increase.

The isolated unpublished lab Agent now selects **active version 2**. The frozen v1 control remains available for comparison; no production channel or release beyond v2 was created.

All twelve cases are public

The machine-readable **evidence/latest.json** (file: evidence/latest.json) includes case_evidence: all 12 original questions and references, both actual response texts, each evaluator's score/Pass/Fail/reason, response-format findings and latency. No difficult case was dropped.

Case	Relevance v1 → v2	TaskAdherence v1 → v2
01 Subscription intent	5 → 5	1 → 1
02 Refund effective-date transition	5 → 5	1 → 1
03 Paid-production-job disqualifier	5 → 5	1 → 1
04 Unsubmitted support inquiry	5 → 5	1 → 1
05 Suspected access incident	5 → 5	1 → 1
06 Separate monthly SLA calculations	5 → 5	1 → 1
07 Beta feature versus production SLA	5 → 5	1 → 1
08 Availability target versus feature claim	5 → 5	1 → 1

Case	Relevance v1 → v2	TaskAdherence v1 → v2
09 Inclusive/exclusive time boundaries	5 → 5	1 → 1
10 Unauthorized tenant deletion	5 → 5	1 → 1
11 Unverified feature and false alternatives	3 → 4	1 → 1
12 Missing SLA calculation inputs	5 → 5	1 → 1

Case 11 improved without pretending to inspect the user's screen or inventing a feature status. V2 gave a clear verdict that **neither asserted conclusion was established**, distinguished missing evidence from absence, and provided an authenticated verification path.

An excerpt from the actual v2 response:

Neither conclusion is established. You don't see a GPU model selection menu, but I haven't inspected your screen or the live service.

The full response and unedited managed-evaluator reasons are in the public JSON. Evidence means actual execution data—not credentials. Tokens, cookies, signed URLs, account identifiers and raw conversation/tool payloads are excluded.

Model roles and instruction provenance

Role	Model/version	Boundary
Agent	gpt-6-sol / 2026-09-22	Actual Agent and knowledge-tool runtime verified; same in v1/v2
Evaluation Judge	gpt-6-luna / 2026-09-22	Used by both real managed runs
Optimizer generator	gpt-5.5 / 2026-04-24	Separate role following the supported optimization-model list

Actual managed Optimizer job: `opt_87805603c0d74b7897f997502aaea080`, instruction-only, at most two generated candidates. The service **retained the strong v1 baseline** rather than recommending either generated candidate.

V2 was an **operator-reviewed refinement after that job**, not a falsely attributed automatic Optimizer promotion. It preserved the strong baseline and clarified how to answer unsupported alternatives with a factual verdict and verification path. It contains no embedded evaluation questions, reference answers, policy

rates or worked dataset examples.

Source: v1 instructions (file: `prompts/en/baseline.txt`) · v2 instructions (file: `prompts/en/optimized.txt`).
Candidate development used explicit drafts; the released workshop Agent has **only versions 1 and 2**.
Immutable versions are not overwritten or silently incremented.

Statistical and operational limits

The real Foundry comparison used **PairedTTest**:

Evaluator	Mean difference	p-value	Service result
Relevance	+0.08333	0.33880	Inconclusive
TaskAdherence	0	1.00000	Inconclusive

The descriptive gain is real in these saved results, but the sample is small and reused during instruction development. Do not claim independent generalization, statistical significance, lower cost or faster operation. The p95 and token increases remain visible rather than being hidden behind the improved pass rate. Actual billing was not observed and is not reported as zero.

`scripts/compare_foundry_eval.py` enforces matched complete results, unchanged conditions, no lower aggregate quality counts/means and at least one strict improvement. It also retains malformed outputs as failures. This is a validation gate over **service-returned scores**, not a new local Judge.

Sources and publication boundaries

Only the latest v2 and its same-model v1 control appear in this current report. Earlier raw records remain in the local audit; they were not relabeled or erased from Azure/Git history.

The capture manifest (file: `web/assets/portal/en/captures.json`) and NOTICE (file: `web/assets/NOTICE.txt`) describe the real portal images. Those earlier images are **UI illustrations only**, not current Sol scoring evidence. Use the current run IDs and full public case record for the measurement.

Official references: [Agent evaluation](https://learn.microsoft.com/azure/foundry/observability/how-to/evaluate-agent) (https://learn.microsoft.com/azure/foundry/observability/how-to/evaluate-agent) · [Agent Optimizer](https://learn.microsoft.com/azure/foundry/agents/quickstarts/quickstart-optimize-prompt-agent) (https://learn.microsoft.com/azure/foundry/agents/quickstarts/quickstart-optimize-prompt-agent) · [Optimization-model roles](https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models) (https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models) · [Evaluator definitions](https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators) (https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators).

Contoso data is synthetic. Earlier design attribution remains at the [archived source commit](https://github.com/junwoojeong100/foundry-evaluation-labs-v0.9/tree/93bc07e31373c4cfc278a2dc3757785946404cf2) (<https://github.com/junwoojeong100/foundry-evaluation-labs-v0.9/tree/93bc07e31373c4cfc278a2dc3757785946404cf2>); it is not another current verification report.

Recheck local deliverables

The documentation revision date is not a new service measurement date. Setup/cleanup improvements and read-only checks on 2026-10-03 are recorded in [execution issues](#). Existing service scores, responses, and evaluator reasons here and in `evidence/latest.json` are not rewritten for a documentation update.

```
python scripts/build_datasets.py --language en --check
python scripts/build_datasets.py --language ko --check
python scripts/build_guide.py --check
python -m unittest discover -s tests -q
```

These commands check files and tests without submitting another Azure evaluation. Historical document/browser/PDF checks in the JSON refer to the deliverables of that time, not the newly generated files. The issue record distinguishes this revision's verification scope.

Synthetic English evaluation data

[Dataset step](#) · [Operator handoff](#) · Korean data guide (file: docs/ko/data-guide.html)

One unchanged dataset for the comparison

Use **data/en/optimizer/dev.jsonl** (file: data/en/optimizer/dev.jsonl), exactly **12 JSONL records**, for Foundry Evaluation, Agent Optimizer, and reevaluation. In a new project, follow [04](#) to register lab-en-dev12 version 1. In a prepared project, reuse its operator-confirmed registration.

Keep its exact bytes and SHA-256, all questions and references, and language fixed. The current Agent is contoso-eval-en-sol, with released instruction variants v1/v2 on the same gpt-6-sol model. The original Korean corpus is separate and has not been newly executed as part of this English measurement.

Purpose and limits

Contoso Atlas Cloud is fictional. Its policies, inquiries and reference answers are synthetic—not private customer data or the terms of Microsoft, Azure or a real provider.

The purpose is **evaluation against your own business tasks and criteria**, not a public benchmark alone. Representative tasks expose policy-date boundaries, ambiguity, unsupported certainty and action claims. A 12-case development set demonstrates the learning loop; it does not certify production or independent generalization.

Preserve the original source corpus

data/en/cases.jsonl (file: data/en/cases.jsonl) contains 100 cases: train 56, validation 12, dev 12 and test 20. Only dev12 is used in this workshop. Other source subsets remain available but are not additional participant exercises.

Do not select only easy rows, duplicate questions, merge subsets, or replace references after seeing results. The eight policy documents (file: data/en/knowledge/documents.json) retain stable ATLAS-* IDs and effective dates. The participant or authorized operator prepares policy access with the [commands in 03](#).

Upload and response schema

Each JSONL line has exactly three columns:

Column	Type	Purpose
query	String	The only Agent input
context	String	Source-policy reference for supported evaluators and case review
ground_truth	JSON string	Structured reference answer, not a generation prompt

There is no prefilled response. Foundry invokes the pinned Agent to obtain one. Do not turn the file into a JSON array, rename columns, append references to the user message or precompute answers.

The reference answer and Agent response use exactly four keys:

Key	Contract
answer	Nonempty English string
citations	Unique stable IDs of supporting policy documents
route	answer, clarify, escalate or refuse
needs_human	Boolean; true exactly when route is escalate

The source response schema (file: `schemas/response.schema.json`) also checks routing consistency. Native structured output controls the JSON shape, not the factual truth of an answer. Numeric retrieval IDs are not policy citations.

The Agent cannot actually submit tickets, change subscriptions, approve credits or delete records. Classification as `escalate` does not mean a person was contacted. A request for unsupported certainty does not authorize fabricated facts.

Register once; pin the same version

Use **Foundry** → **Build** → **Evaluations** → **Create** → **Agent**, an explicit baseline version, **Individual turns / One time / Existing dataset**. Reselect the target checkbox if version selection clears it.

Choose the existing dev12 registration. In a fresh authorized project only, upload the exact language-specific file and record its new dataset version. The preview may show five rows; the file and evaluation scope still contain 12.

Leave the custom prompt override unset. Use `query → query` if a field-mapping screen appears. Service mappings are **Relevance** `response={{sample.output_text}}` and **TaskAdherence** `response={{sample.output_items}}`. These are not extra JSONL columns.

Keep **Relevance 1–5 / threshold 4**, **TaskAdherence binary 0/1 / pass 1**, and the same Luna Judge. Selecting two evaluators does not mean both consume every reference field or certify every business rule.

Compare complete evidence

The [latest report](#) publishes the current v2 and its frozen same-model v1 control, including all 12 synthetic questions, actual answers, evaluator scores and reasons. Older raw runs are retained for audit, not mixed into the current comparison.

The helper checks matching questions/references and actual per-item Agent version/instructions. Missing/error rows cannot become successful rows or disappear from the denominator. JSON/route checks are supplementary validation, **not a replacement local Judge**.

Agent Optimizer's internal ranking is distinct from the separate managed run's means and pass counts. Review candidate instructions and preserve model/tools/reasoning/schema. Compare one reviewed v2 without continually adding releases. Operators may use separate drafts after verifying support.

Public evaluation evidence excludes authentication, signed URLs and private account metadata. The synthetic responses and failure reasons themselves may be shared. Publish failures as faithfully as successes.

`cleanup` does not remove every registered dataset. Follow [10](#) to retain or delete registrations/evaluation records according to dedicated or shared-project ownership.